

Gaza Strip Dijkstra Project

Instructor Discussion Guide Data Structures & Algorithms — Project 3

1. Project Overview

This project implements a shortest-path visualizer for the Gaza Strip road network. A weighted undirected graph is built from a text file containing city coordinates and adjacency information. Dijkstra's algorithm with a custom binary min-heap finds the shortest path between any two cities. The result is displayed on a georeferenced OpenStreetMap tile background using a JavaFX canvas.

Component	Implementation
Graph	Adjacency-list (DynArray<Edge>[])
Heap	Binary min-heap with pos[] for O(1) contains/decreaseKey
Name lookup	Open-addressed hash map (SimpleHashMap)
Distance	Haversine formula — computed at runtime, never stored
Map projection	Web Mercator (matches OSM tiles exactly)
UI	JavaFX Canvas + ComboBox — dark-mode theme

2. How Vertices Are Stored

2.1 The City Class

Each vertex is a `City` object holding its name, GPS coordinates, and two runtime pixel coordinates used for rendering on the map canvas:

```
public class City { public final String name; public final double lat, lon; public
final boolean isIntersection; // name starts with "Int_" public double x, y; // pixel
coords (set at runtime) }
```

2.2 Plain Array vs ArrayList

All vertices are stored in a plain Java array: `City[] cities`. The size is read from the first line of the file, so the array is allocated exactly once with no resizing needed.

- **O(1) random access** by integer index — every graph operation references a city as `cities[i]`.
- **No boxing overhead** — `City[]` stores object references directly; no wrapper types.
- **Cache-friendly iteration** — contiguous memory layout for the reference array.
- **Fixed size is fine** — the vertex count is known before the array is created.
- `java.util.ArrayList` wraps an `Object[]` internally and adds bounds-checking and autoboxing cost that is unnecessary when the size is already known.

2.3 DynArray — Custom Dynamic Array

Edges and the touched-vertex list require a growable container. We implemented `DynArray<T>` from scratch to replace `java.util.ArrayList`:

```
public class DynArray<T> { private Object[] data; // internal array, doubles when full
private int size; // add O(1) amortized | get O(1) | clear O(1) }
```

The adjacency list is declared as `DynArray<Edge>[] adj` — one `DynArray` per vertex. Each `DynArray` starts with capacity 4 and doubles when full, giving $O(1)$ amortised append.

3. How the File Is Read

3.1 File Format

```
37 63 # numCities numEdges Gaza 31.5017 34.4667 Beit_Lahiya 31.5500 34.4950 ... # one
city per line Gaza Beit_Lahiya # edge: two city names ...
```

3.2 GraphLoader.load()

A single `BufferedReader` pass reads the file from top to bottom. Lines beginning with `#` and blank lines are skipped by the helper `nextLine()`. The entire parse is $O(V + E)$ time and $O(V)$ space.

```
// Step 1 – parse header int numCities = ..., numEdges = ...; // Step 2 – create City
objects + fill SimpleHashMap for (int i = 0; i < numCities; i++) { cities[i] = new
City(name, lat, lon); nameMap.put(name, i); } // Step 3 – resolve edge names to
indices, call addEdge() for (int i = 0; i < numEdges; i++) { int u =
nameMap.get(partA); // O(1) int v = nameMap.get(partB); // O(1) graph.addEdge(u, v);
// computes Haversine, stores Edge }
```

3.3 SimpleHashMap — $O(1)$ Name Lookup

City names are resolved to graph indices via a hand-built open-addressed hash map. No `java.util.HashMap` is used. The hash function is:

```
private int slot(String key) { int h = (key.hashCode() & 0x7fffffff) % cap; while
(keys[h] != null && !keys[h].equals(key)) h = (h + 1) % cap; // linear probing return
h; }
```

Capacity is set to $2 \times \text{numCities}$, giving a load factor of ~ 0.5 and very few collisions. Average lookup is $O(1)$; worst-case is $O(V)$ but never reached in practice at this load factor.

4. How Distances Are Stored and Why Not Pre-Stored

4.1 Haversine — Computed at Runtime

Edge weights are **not written in the input file**. They are computed once during graph construction by the **Haversine formula**, which gives the great-circle distance in kilometres between two GPS coordinates:

```
public static double haversine(City a, City b) { final double R = 6371.0; // Earth
radius km double dLat = toRadians(b.lat - a.lat); double dLon = toRadians(b.lon -
a.lon); double h = sin(dLat/2)*sin(dLat/2) +
cos(toRadians(a.lat))*cos(toRadians(b.lat)) * sin(dLon/2)*sin(dLon/2); return
2*R*asin(sqrt(h)); }
```

The result is stored in `Edge.weight` at graph-build time. Each edge is computed **exactly once**; Dijkstra reads `e.weight` directly in $O(1)$.

4.2 Why Not Store Distances in the File?

- The file stays compact — only city names appear on edge lines.
- Distances derived from real lat/lon are more accurate and reproducible than hand-entered km values.
- Adding a new city only requires its coordinates; all distances to it are computed automatically.
- No risk of stale or inconsistent pre-computed values.

5. Dijkstra's Algorithm — Implementation & Optimisations

5.1 MinHeap — Binary Min-Heap from Scratch

The priority queue is a 1-indexed binary min-heap backed by a `Node[]` array. A companion `pos[]` array maps each city index to its current position in the heap, enabling $O(1)$ lookup for `contains()` and $O(\log V)$ `decreaseKey()`:

```
private Node[] heap; // 1-indexed private int[] pos; // pos[cityIdx] = position in
heap, -1 if absent void decreaseKey(int cityIdx, double newDist) { int i =
pos[cityIdx]; // O(1) lookup heap[i].dist = newDist; bubbleUp(i); // O(log V) }
```

5.2 Early Termination

Standard Dijkstra settles every reachable vertex. Our implementation breaks out of the main loop the moment the destination vertex is settled:

```
MinHeap.Node node = pq.deleteMin(); int u = node.cityIdx; settled[u] = true; if (u ==
dst) break; // done – no need to explore further
```

For a thin strip like Gaza, this reduces explored vertices significantly for distant city pairs.

5.3 Lazy Reset Between Queries

Instead of resetting `dist[]`, `prev[]`, and `settled[]` across all V vertices before each new query ($O(V)$), we track only the vertices *touched* by the previous query in a `DynArray<Integer> touched`. The reset loop runs in $O(k)$ where $k \ll V$:

```
for (int i = 0; i < touched.size(); i++) { int idx = touched.get(i); dist[idx] =
Double.MAX_VALUE; prev[idx] = -1; settled[idx] = false; } touched.clear();
```

6. Time & Space Complexity Summary

Operation	Time	Space	Notes
Graph build (V cities, E edges)	$O(V + E)$	$O(V + E)$	Haversine per edge at build time
Dijkstra full run	$O(E \log V)$	$O(V)$	Binary min-heap
Early termination (src→dst)	$O(E' \log V)$	$O(V)$	Stops when dst settled
Lazy reset between queries	$O(k)$	—	k = touched vertices only
decreaseKey in heap	$O(\log V)$	—	<code>pos[]</code> gives $O(1)$ index lookup
City name lookup (SimpleHashMap)	$O(1)$ avg	$O(V)$	Open-addressed, load 0.5
Haversine distance	$O(1)$	$O(1)$	6 trig ops, no storage
File parse	$O(V + E)$	$O(V)$	Single buffered pass

7. Map — OSM Tiles & Web Mercator Projection

7.1 Why Tiles Instead of OSM Export

The openstreetmap.org/export page downloads raw node/way XML. The Gaza Strip has far more than the 50,000-node API limit, producing an error. Our **MapDownloader** class fetches pre-rendered 256×256 PNG tile images instead — these have no node limit and are stitched into a single background image.

7.2 Web Mercator Projection

OSM tiles use the Web Mercator projection (EPSG:3857). A naive linear (equirectangular) mapping of latitude would misalign city dots because Mercator compresses latitudes near the equator. We use the exact formula:

```
private double mercatorY(double lat) { double r = Math.toRadians(lat); return
Math.log(Math.tan(r) + 1.0 / Math.cos(r)); } // Project city to canvas pixels: c.x =
margin + (c.lon - minLon) / lonRange * usableW; double mY = mercatorY(c.lat); c.y =
margin + (1.0 - (mY - mercYMin) / mercRange) * usableH;
```

8. Anticipated Instructor Questions & Answers

Q: Why did you use a plain array for cities instead of ArrayList?

A: The city count is read from the first line of the file before any City object is created, so we can allocate `City[] cities = new City[numCities]` exactly once with no resizing. This gives $O(1)$ random access with no boxing overhead. `ArrayList` would add unnecessary autoboxing and an extra layer of indirection.

Q: What is DynArray and why did you build it instead of using ArrayList?

A: `DynArray` is our custom dynamic array built over `Object[]`. It doubles its capacity when full, giving $O(1)$ amortised append, $O(1)$ get, and $O(1)$ clear — the same asymptotic behaviour as `ArrayList` but with no dependency on `java.util` and full transparency over the internal structure.

Q: How does the Haversine formula work and why isn't the distance stored in the file?

A: Haversine computes great-circle distance from two lat/lon pairs using the formula $d = 2R * \arcsin(\sqrt{\sin^2(dLat/2) + \cos(lat1)\cos(lat2)\sin^2(dLon/2)})$. Distances are computed once at graph-build time and stored in `Edge.weight`. They are not in the file because (a) the file stays smaller, (b) computed values are always consistent with the coordinates, and (c) adding a new city only requires its lat/lon.

Q: What is the time complexity of your Dijkstra and what optimisations did you apply?

A: $O(E \log V)$ with a binary min-heap. Two optimisations: (1) Early termination — the loop breaks as soon as the destination vertex is settled, so we never explore vertices farther than the destination. (2) Lazy reset — between queries we only reset the vertices that were actually touched by the previous query (stored in `DynArray touched`), reducing inter-query cleanup from $O(V)$ to $O(k)$ where k is the number of settled/relaxed vertices.

Q: What does the `pos[]` array in `MinHeap` do?

A: `pos[cityIdx]` stores the current position of that city in the heap array. This gives $O(1)$ lookup for `contains()` (just check `pos[i] != -1`) and lets `decreaseKey()` jump directly to the node's position and `bubbleUp` in $O(\log V)$ without scanning the heap. Without `pos[]`, `decreaseKey` would require $O(V)$ linear search.

Q: How does SimpleHashMap work and why not use java.util.HashMap?

A: `SimpleHashMap` is an open-addressed hash map over parallel `String[]` keys and `int[]` vals arrays. The `slot()` method computes $h = hashCode \% cap$, then linear-probes on collision. Capacity is $2 * numCities$ (load factor ~ 0.5), giving $O(1)$ average insert and lookup. We avoid `java.util.HashMap` to demonstrate the underlying mechanism and eliminate boxing of int values into Integer objects.

Q: How do intersection nodes differ from city nodes?

A: Any city whose name starts with 'Int_' is flagged as `isIntersection = true` in the `City` class. Intersection nodes are added to the graph and participate in Dijkstra exactly like city nodes, but they are excluded from the `ComboBox` dropdowns and from the path log display. This lets the graph model real road junctions for accurate routing while keeping the UI focused on named cities.

Q: Why is the projection order `buildIndexMaps` → `computeMapBounds` → `loadMapImage` → `projectCities` important?

A: `projectCities()` maps lat/lon to canvas pixels using `minLat/maxLon` bounds that are set either from `MapDownloader.computeExactBounds()` (after tile download) or from defaults. `loadMapImage()` must run first so those bounds are known. If `projectCities()` ran before `loadMapImage()`, it would use the approximate defaults, causing city dots to be misaligned on the downloaded tile image.